

Data-Flow Testing and Coverage Concepts

SE 413/513 — Software Testing

Course Reading Material

Unit: White-Box Testing and Structural Coverage Criteria

© Dr. Kamrul Hasan, Grand Valley State University. All rights reserved.

For permissions or questions regarding the use of this material, contact hasanka@gvsu.edu.

Please cite [1] when using, adapting, or reproducing this material. The corresponding BibTeX entry is provided below:

```
@misc{hasan2026stesting,  
author = {Hasan, Kamrul},  
year = {2026},  
title = {Software Testing},  
institution = {Grand Valley State University},  
url = {https://github.com/mdkamrulhasan/software-testing}  
}
```

Contents

1	Learning Objectives	3
2	From Control Flow to Data Flow	3
3	Definitions and Uses	3
4	Why Data-Flow Testing Matters	4
5	Def-Use Relationships: Def-Clear Paths and DU Pairs	5
6	A Worked Example: <code>sum_positive()</code>	5
6.1	The Function	5
6.2	Definitions and Uses	6
6.3	Annotated Control-Flow Graph	6
6.4	Def-Use Pairs	7
6.5	Test Cases and Coverage	8
7	Data-Flow Coverage Criteria	8
7.1	How the Criteria Relate	9
8	Data-Flow Testing in Practice	9
9	Common Pitfalls	10
10	A Practical Mental Model	10

11 Summary	11
12 Practice Exercises	11
13 Discussion Questions	12
14 Key Takeaways for SE 413/513	12
15 References	13

1 Learning Objectives

After completing this reading, you should be able to:

- Explain what data-flow testing is and how it differs from purely control-flow-based coverage.
- Identify definitions (defs) and uses of a variable in a piece of code.
- Distinguish a computation-use (c-use) from a predicate-use (p-use).
- Explain what a def-clear path is and why it matters.
- Identify the def-use (DU) pairs for a small function, including a loop-carried pair.
- Define the All-Defs, All-Uses, and All-DU-Paths coverage criteria.
- Explain how the data-flow criteria relate to each other and to statement and branch coverage.
- Design a small set of test cases that achieves a target data-flow coverage criterion.

2 From Control Flow to Data Flow

The *Control-Flow Graphs and Coverage* reading showed how statement, branch, and path coverage measure testing thoroughness in terms of *which parts of the code ran*. Those criteria are entirely about control: which statements executed, which branch outcomes were taken, which paths through the graph were exercised. They say nothing about what happened to the *data* moving through that code.

This matters because many real defects are not control-flow defects at all. A function can execute every statement and every branch and still compute the wrong answer because a variable was read before it was properly set, because a value was overwritten before anything used it, or because a stale value from an earlier iteration leaked into a later computation. These are *data-flow* defects, and control-flow coverage is structurally blind to them: a statement can be “covered” by a test that runs it while using entirely the wrong value.

Data-flow testing is a white-box technique that designs test cases around how variables are *defined* (assigned a value) and *used* (read) as the program executes, rather than around which statements or branches merely execute. Instead of asking “did this line run?”, data-flow testing asks “did a value that was assigned here actually reach, unchanged, a place that uses it?”

Key Idea

Control-flow coverage asks *which code ran*. Data-flow coverage asks *whether the right values reached the right places*. The two are complementary, not competing: a mature test suite typically uses both.

3 Definitions and Uses

Data-flow testing is built on two basic events that can happen to a variable at a given statement:

- A **definition (def)** is a statement where a variable is assigned a value: an assignment, a value read from input, a loop variable being bound, or a parameter being passed in.
- A **use** is a statement where a variable’s current value is read.

Uses are further split into two kinds, because they play different roles in testing:

- A **computation-use (c-use)** is a use where the variable’s value contributes to a computation or output: appearing on the right-hand side of an assignment, in an expression whose result is stored or returned, or as an argument to a function call.

- A **predicate-use (p-use)** is a use where the variable’s value controls which branch is taken: appearing in the condition of an `if`, `while`, `for`, or similar construct.

Event	Meaning	Example
Definition (def)	Variable is assigned a value	<code>total = 0</code>
c-use	Value is read for a computation or output	<code>return total</code>
p-use	Value is read to choose a branch	<code>if n > 0:</code>

A single statement can both use and define a variable, and can even define one variable while using another. In `total = total + n`, the statement is a c-use of `total` (its old value is read), a c-use of `n`, and a definition of `total` (its new value is stored) — all at once.

Tip

A useful habit when reading unfamiliar code: for every variable, mentally tag each statement that touches it as “D” (def), “P” (p-use), or “C” (c-use). This is exactly what static data-flow analysis tools do automatically, and it is the same annotation this reading will attach to a control-flow graph in Section 6.

4 Why Data-Flow Testing Matters

Data-flow testing is aimed squarely at a defect class that statement and branch coverage cannot see: a variable’s value being wrong even though the code that touches it ran. Three recurring patterns motivate it:

- **Use before def.** A variable is read before it has been meaningfully assigned on that path — for example, an error-handling branch that references a result variable that was never set on the path that reached it. Depending on the language, this can silently return garbage, a default, or a leftover value from an earlier call, rather than crashing.
- **Dead (unused) definitions.** A variable is assigned a value that is never read before being overwritten or going out of scope. This is often harmless dead code, but it can also indicate a forgotten update elsewhere — the assignment exists because the programmer intended it to matter, and the fact that nothing reads it is itself a signal something is missing.
- **Definitions overwritten before use (“killed” definitions).** A value is computed and then clobbered by a second assignment before anything reads the first one. If the second assignment is a bug — happening on a path where it should not — control-flow coverage can be 100% while the program silently uses the wrong value.

All three patterns share a signature: the *code* executed exactly as intended (control-flow coverage is satisfied), but the *data* did not flow the way the programmer assumed. Data-flow testing exists to make that assumption explicit and testable.

Real-World Case Study

The Debian OpenSSL Weak-Key Bug (CVE-2008-0166, 2006–2008).^[4] In 2006, a Debian maintainer patched OpenSSL’s random-number generator to silence Valgrind warnings about reading uninitialized memory. The catch: two calls that fed apparently “uninitialized” buffer contents into the entropy pool,

```
MD_Update(&m, buf, j);
```

were commented out in `ssleay_rand_add()` inside `crypto/rand/md_rand.c`. That memory was uninitialized *on purpose* — its unpredictable contents were part of the entropy source. Removing those two lines meant the only remaining source of randomness feeding the seed was the process ID, which on Linux ranges only from 0 to 32,767. Every key, certificate, and token generated by the patched OpenSSL on Debian- and Ubuntu-based systems between 2006 and May 2008 came from a keyspace of at most 32,768 possible values — trivially brute-forceable. The flaw affected SSH host and user keys, SSL/TLS certificates, OpenVPN keys, DKIM keys, and session tokens across affected systems, and required mass key regeneration across the Internet once DSA-1571-1 disclosed and fixed it in May 2008.

From a data-flow perspective, the deleted lines were *definitions* that fed the entropy pool. The maintainer’s static analyzer (Valgrind) flagged a *use* of uninitialized memory as suspicious — correctly, from a memory-safety standpoint, but incorrectly from a data-flow standpoint, because that “uninitialized” read was the intended definition source. No amount of statement or branch coverage on the patched code would have revealed the problem: every line that remained ran exactly as written. The defect was entirely about which values reached the seed, not which statements executed.

5 Def-Use Relationships: Def-Clear Paths and DU Pairs

Data-flow testing centers on a single relationship: does a value assigned at one point in the code actually reach a particular use, unchanged, along some executable path?

A path from a definition of variable x to a use of x is **def-clear** (with respect to x) if no other definition of x occurs anywhere along that path between the two. If the path is def-clear, the value read at the use is guaranteed to be the value written at that definition — that specific def genuinely reaches that specific use.

A **def-use (DU) pair** is a triple consisting of a variable, a definition of that variable, and a use of that variable, connected by at least one def-clear path. Formally: (x, d, u) is a DU pair if d is a definition of x , u is a use of x , and some def-clear path from d to u exists in the control-flow graph.

DU pairs are the atomic unit that data-flow coverage criteria count. Where branch coverage asks you to exercise every edge in the control-flow graph, data-flow coverage asks you to exercise def-use *relationships* — and, as the worked example below shows, a single variable inside a loop can have DU pairs that only become reachable across multiple loop iterations.

Common Pitfall

A definition and a later use of the same variable do *not* automatically form a DU pair. If another definition of that variable occurs anywhere on every path between them, the original value never survives to reach that use — it was *killed* first. Only a genuinely def-clear path creates a DU pair.

6 A Worked Example: `sum_positive()`

6.1 The Function

Consider a small function that sums the positive numbers in a list:

```

def sum_positive(numbers):
    total = 0                # S1
    for n in numbers:        # S2
        if n > 0:             # S3
            total = total + n # S4
    return total              # S5

```

This function is deliberately simple, but it contains everything needed to illustrate the core ideas: a variable defined once before a loop (`total`), a variable redefined inside the loop using its own previous value, a loop variable freshly defined on every iteration (`n`), a predicate-use that controls a branch, and — because `total` is both defined and used on statement S4 — a definition that can reach a use in a *later* iteration of the same loop.

6.2 Definitions and Uses

Stmt	Code	Def/Use Events
S1	<code>total = 0</code>	def <code>total</code> (call it D1)
S2	<code>for n in numbers:</code>	def <code>n</code> (fresh each iteration)
S3	<code>if n > 0:</code>	p-use <code>n</code>
S4	<code>total = total + n</code>	c-use <code>total</code> , c-use <code>n</code> ; def <code>total</code> (call it D2)
S5	<code>return total</code>	c-use <code>total</code>

Two definitions of `total` exist: D1 at S1 (the initial value) and D2 at S4 (the running-sum update). D2 is special because S4 both reads and rewrites `total` — it is simultaneously a use of the *previous* value and a definition of a *new* value.

6.3 Annotated Control-Flow Graph

Figure 1 overlays the def/use events from the table above onto the function’s control-flow graph, the same style of diagram used in the *Control-Flow Graphs* reading. Notice the two back-edges into the loop-condition node: one from S4 (after the sum is updated) and one from the false outcome of S3 (when an element is skipped). Either back-edge can carry D2’s new value of `total` into a later pass through S4.

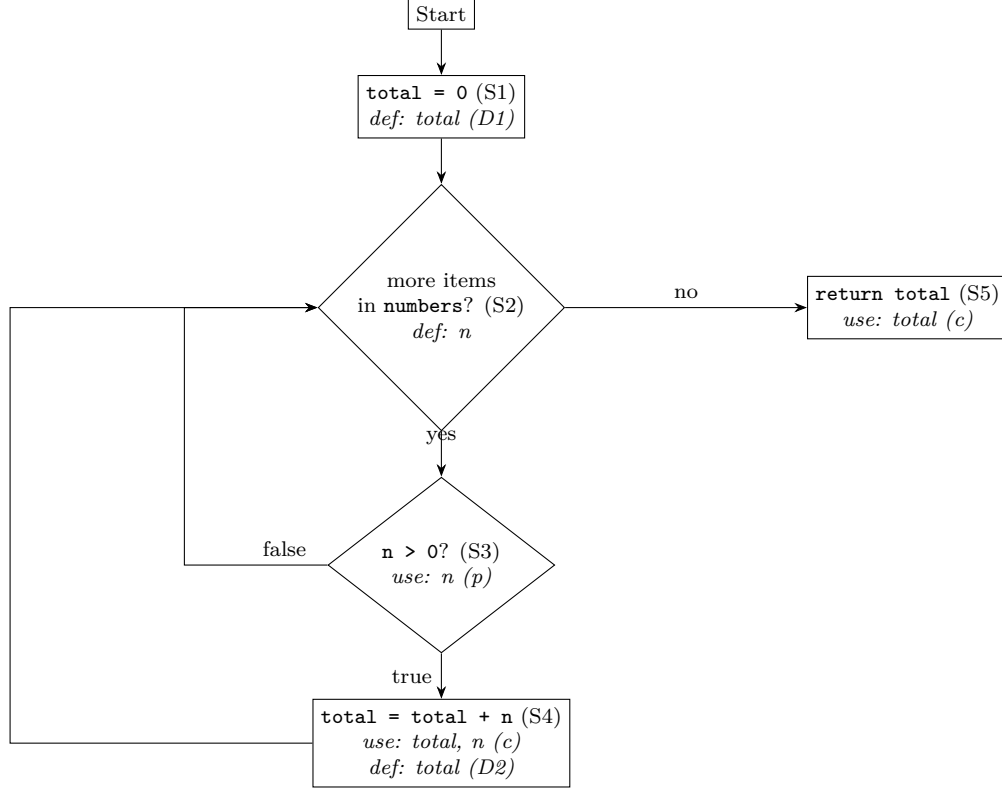


Figure 1: Control-flow graph for `sum_positive()`, annotated with def/use events. (c) marks a computation-use, (p) marks a predicate-use.

6.4 Def-Use Pairs

Tracing def-clear paths through Figure 1 gives the following DU pairs. Pairs are grouped by variable.

#	Var	Def	Use	Notes
1	total	D1 (S1)	S4 (c-use)	first time through the loop
2	total	D1 (S1)	S5 (c-use)	def-clear when S4 never executes (e.g., empty list, or no element > 0)
3	total	D2 (S4)	S4 (c-use)	loop-carried: D2 from one iteration reaches S4 on a <i>later</i> iteration, if untouched between them
4	total	D2 (S4)	S5 (c-use)	def-clear when S4 executes at least once, then the loop exits with no further redefinition
5	n	def (S2)	S3, true (p-use)	every iteration where the element is > 0
6	n	def (S2)	S3, false (p-use)	every iteration where the element is ≤ 0
7	n	def (S2)	S4 (c-use)	only reachable when the S3 predicate is true

Pair #3 is the one genuinely new idea this example introduces beyond the control-flow reading: it is a DU pair whose definition and use are the *same statement*, connected across a loop back-edge. No single pass through the function’s body creates it — it only exists because `total` survives, unmodified by anything else, from the end of one iteration into the start of a later one.

Key Idea

A definition and a use do not need to appear in that order in the source text. A definition at the *end* of a loop body can be the source for a use at the *same* statement, or even earlier in the loop body, on the next iteration — because control flows back around the loop. This is exactly what pair #3 captures.

6.5 Test Cases and Coverage

Three small test cases exercise every DU pair listed above:

Input	Result	New pairs covered
[-1]	0	#2 (D1→S5); #6 (n def→p-use false)
[3]	3	#1 (D1→S4); #4 (D2→S5); #5 (n def→p-use true); #7 (n def→S4)
[3, 5]	8	#3 (D2→S4, loop-carried)

These three inputs were verified by direct execution against the function above, returning 0, 3, and 8 respectively, matching the **Result** column. Together they cover all seven DU pairs in the table above — this is exactly the goal of the All-Uses criterion defined next.

Tip

Notice that [-1] and [3] together already give every static definition in the function at least one reachable use (D1 reaches S4 in test 2, D2 reaches S5 in test 2, and n’s definition reaches both S3 outcomes and S4). Only pair #3 — the loop-carried pair — needs a *third* test with two positive elements. Section 7 uses this exact gap to distinguish two of the coverage criteria.

7 Data-Flow Coverage Criteria

Just as statement, branch, and path coverage give increasingly strong control-flow-based criteria, data-flow testing defines a family of criteria of increasing strength, based on how thoroughly DU pairs must be exercised.[3] The three most commonly taught are due to Rapps and Weyuker’s original data-flow coverage family:[3]

- **All-Defs coverage.** For every definition of every variable, at least one def-clear path from that definition to *some* use is exercised. It does not matter which use — only that each definition reaches at least one of its possible uses.
- **All-Uses coverage.** For every definition of every variable, a def-clear path to *every* use reachable from that definition is exercised. Every DU pair in the sense of Section 5 is covered at least once.
- **All-DU-Paths coverage.** For every DU pair, *every* distinct def-clear path connecting that definition to that use is exercised, not merely one. Where All-Uses asks “was this def-use relationship exercised at all,” All-DU-Paths asks “were all the different ways it could happen exercised.”

Applied to `sum_positive()`: the test cases [-1] and [3] together achieve All-Defs coverage, because every definition (D1, D2, and n’s definition) reaches at least one use. They do *not* achieve All-Uses coverage, because pair #3 (D2 looping back to S4) is still uncovered. Adding [3, 5] closes that gap and achieves All-Uses coverage.

The same three tests do *not* achieve All-DU-Paths coverage, even in this small function, because the loop gives some DU pairs more than one def-clear path. In [3, 5], pair #3 travels from S4 straight back through the loop condition to S3 (true) and S4. In [3, -2, 5], the same pair travels along a different def-clear path:

from S4 around the loop, through S3’s *false* outcome for the skipped element -2, around the loop again, and only then through S3 (true) to S4. Nothing redefines `total` along the way, so D2 still reaches S4 — by a path [3, 5] never takes. Pair #2 behaves the same way: D1 reaches S5 directly for an empty list, or after one or more skipped elements, as in [-1]. Every additional skipped element produces yet another distinct def-clear path, which is exactly why All-DU-Paths is unbounded for this loop (see the warning box below).

Criterion	Requirement
All-Defs	Every definition reaches at least one use (some use, not necessarily all)
All-Uses	Every definition reaches every use it can reach — every DU pair covered
All-DU-Paths	Every distinct def-clear path for every DU pair is covered, not just one

7.1 How the Criteria Relate

The data-flow criteria form a subsumption hierarchy among themselves, and they connect to the control-flow criteria from the earlier reading:

- **All-DU-Paths $\Rightarrow$ All-Uses $\Rightarrow$ All-Defs.** Each is strictly stronger than the last: exercising every path for every pair certainly exercises every pair at least once, and exercising every pair certainly gives every definition at least one use.
- **All-Uses $\Rightarrow$ branch coverage $\Rightarrow$ statement coverage.** Every branch in a function corresponds to a predicate-use of some variable, so exercising every DU pair involving a p-use necessarily exercises both outcomes of every branch — which in turn executes every statement.
- **All-Defs does *not* generally subsume branch coverage.** All-Defs is satisfied as soon as each definition reaches *one* use, which can happen without ever exercising both outcomes of a nearby branch. In `sum_positive()`, [-1] and [3] together satisfy All-Defs, and in this particular function they happen to also cover both outcomes of S3 — but that is a coincidence of this small example, not a guarantee of the criterion. In general, a minimal All-Defs-satisfying test set can leave whole branches unexercised.

Common Pitfall

All-DU-Paths, like full path coverage from the *Control-Flow Graphs* reading, can be **infeasible** for looping code: a loop-carried DU pair such as #3 above technically has one distinct def-clear path per possible number of intervening iterations, so demanding literally every one of them is unbounded in general. In practice, teams treat All-DU-Paths as an aspirational upper bound and use All-Uses, bounded-iteration testing (zero, one, many — the same heuristic used for path coverage), or tool-assisted analysis instead of pursuing it exhaustively.

8 Data-Flow Testing in Practice

Unlike statement and branch coverage, which are supported by mainstream coverage tools[2] (`coverage.py`, `gcov`, `JaCoCo`, and similar), general-purpose data-flow coverage tooling is far less common in everyday industry workflows. Computing DU pairs and def-clear paths requires real static data-flow analysis, not just instrumentation of executed lines — closer in spirit to what a compiler’s optimizer or a tool like Valgrind performs than to a simple execution counter.

Where data-flow analysis *is* pervasive in practice is in a closely related family of tools: static analyzers and compilers routinely perform def-use analysis internally to catch use-before-def errors, flag genuinely dead (unused) assignments, and detect variables that are always overwritten before being read. Modern IDEs

surface many of these warnings directly (“local variable is never used,” “variable might be uninitialized”). Memory-checking tools such as Valgrind perform an even more precise, runtime version of this analysis — which is precisely what flagged the OpenSSL entropy reads in Section 4’s case study as suspicious, illustrating both the power and the limits of automated data-flow reasoning: it correctly found a data-flow anomaly, but the human response to that finding was itself the defect.

For test design, the practical takeaway is usually not “run an All-DU-Paths tool on every module,” but rather to *think* in def-use terms when a function has non-trivial internal state: variables that persist and update across loop iterations, accumulator patterns, and any variable whose value at one point depends on a chain of earlier updates are exactly where data-flow bugs hide, and are worth deliberately tracing by hand the way Section 6 did.

9 Common Pitfalls

Common Pitfall

Confusing a def-use pair with a def followed later by a use in the source text. Two occurrences of a variable being merely close together in the code does not create a DU pair unless a def-clear path actually connects them. A definition can be killed by another assignment before the “obvious” use is reached.

Common Pitfall

Assuming All-Defs is enough. As Section 7.1 showed, All-Defs can be satisfied while leaving entire DU pairs — including loop-carried ones — completely untested. All-Defs is a useful minimum, not a thorough criterion.

Common Pitfall

Overlooking loop-carried pairs. It is easy to design tests using only single-pass reasoning (one iteration at a time) and miss DU pairs that only exist because a value survives from one iteration to a later one. Whenever a variable is both defined and used inside a loop body, ask explicitly: does this definition also reach a use in a *later* iteration?

10 A Practical Mental Model

When you see a variable assigned inside a loop, think:

“Where does this value go next — does it get used again this iteration, does it get overwritten, or does it survive to the next iteration? If it survives, that is a DU pair my tests need to exercise deliberately.”

When a coverage report shows 100% branch coverage on a function with an accumulator or running total, think:

“Branch coverage confirms every decision ran both ways. It says nothing about whether the accumulated value was ever actually checked after more than one update.”

When reviewing a bug where “the code ran but produced the wrong value,” think:

“Which definition of the variable in question actually reached the point where it went wrong — and was that the definition the author intended?”

11 Summary

Term	Meaning
Definition (def)	A statement that assigns a value to a variable
Use	A statement that reads a variable's value
c-use / p-use	A use in a computation/output vs. a use that controls a branch
Def-clear path	A path from a def to a use with no intervening redefinition of that variable
DU pair	A (variable, def, use) triple connected by at least one def-clear path
All-Defs	Every definition reaches at least one use
All-Uses	Every DU pair is exercised at least once
All-DU-Paths	Every distinct def-clear path for every DU pair is exercised

Key Idea

Control-flow coverage tells you which code ran; data-flow coverage tells you whether the right values got where they needed to go. Identify each variable's definitions and uses, trace def-clear paths between them to find DU pairs, pay special attention to definitions and uses that span loop iterations, and pick the criterion (All-Defs, All-Uses, or All-DU-Paths) that matches how much confidence the code's complexity warrants — keeping in mind, as with path coverage in the *Control-Flow Graphs* reading, that the strongest criterion is not always feasible to pursue exhaustively.

12 Practice Exercises

Exercise 1 — Identifying Defs and Uses

For the function below, list every statement that defines or uses `result`, and classify each use as a c-use or a p-use:

```
def first_negative(values):
    result = None
    for v in values:
        if result is None and v < 0:
            result = v
    return result
```

Exercise 2 — Finding DU Pairs

For `first_negative()` above, identify all DU pairs for `result`, including any loop-carried pair. Is there a DU pair analogous to pair #3 from the `sum_positive()` example? Explain why or why not.

Exercise 3 — Minimal Test Sets

Design a minimal set of test inputs for `first_negative()` that achieves All-Uses coverage on the `result` variable. Show which pair each test covers.

Exercise 4 — All-Defs vs. All-Uses

Using your answer to Exercise 3, find a smaller test set (fewer test cases) that achieves All-Defs coverage but not All-Uses coverage. Explain precisely which DU pair it misses.

Exercise 5 — Design Question

A teammate says: “Our function has 100% branch coverage, so there’s no need to think about data flow separately.” Using at least one concept from this reading (the subsumption relationships in Section 7.1, the loop-carried pair in the worked example, or the OpenSSL case study), write a short argument explaining what this claim gets right and what it misses.

13 Discussion Questions

1. What is the difference between a c-use and a p-use, and why does that distinction matter for relating data-flow coverage to branch coverage?
2. What does it mean for a path to be “def-clear,” and why is that requirement necessary for a DU pair to exist?
3. Why can a statement both use and define the same variable? Give an example distinct from `total = total + n`.
4. Explain, in your own words, why pair #3 in the `sum_positive()` example only exists because of the loop back-edge.
5. Why does All-Defs coverage not generally guarantee branch coverage, even though the worked example’s minimal All-Defs test set happened to cover both branch outcomes?
6. Why can All-DU-Paths coverage be infeasible for code containing loops, and how is that similar to the infeasibility of full path coverage?
7. In the Debian OpenSSL case study, which def-use relationship was broken, and why was it invisible to statement and branch coverage?
8. Why is general-purpose data-flow coverage tooling less common in everyday industry practice than statement or branch coverage tooling?
9. Give an example (not from this reading) of a real or plausible bug that a use-before-def analysis would catch but that 100% statement coverage would not.
10. How do data-flow coverage criteria and the control-flow criteria from the *Control-Flow Graphs* reading complement each other in a real test strategy?

14 Key Takeaways for SE 413/513

1. **Data-flow testing targets a defect class control-flow coverage cannot see:** wrong values reaching a computation even though every relevant statement and branch executed.
2. **Every variable touch is a def, a c-use, or a p-use** — and a single statement can be more than one of these at once.
3. **A DU pair requires a def-clear path**, not just textual proximity between a definition and a later use.
4. **Loops can create DU pairs that span iterations.** A definition at the end of a loop body can reach a use on a later pass through the same statement.
5. **All-Defs, All-Uses, and All-DU-Paths form a strict hierarchy**, with All-Defs the weakest and easiest to satisfy accidentally.
6. **All-Uses subsumes branch and statement coverage; All-Defs generally does not** — a subtlety worth remembering when a teammate claims high branch coverage is “enough.”
7. **Real-world impact is not hypothetical:** the Debian OpenSSL entropy bug shows a single broken data-flow relationship compromising cryptographic keys across the Internet, in code that executed exactly as its (broken) logic intended.

15 References

- Course material on software testing principles and related testing practices.[1]
- White-box testing and structural coverage concepts.[2]
- Foundational data-flow test-data selection and coverage criteria.[3]
- Official documentation of the Debian OpenSSL weak-key vulnerability (CVE-2008-0166).[4]

References

- [1] Hasan, K. (2026). *Software Testing*. Grand Valley State University. <https://github.com/mdkamrulhasan/software-testing>
- [2] International Software Testing Qualifications Board (ISTQB). (2024). *Certified Tester Foundation Level (CTFL) Syllabus, v4.0.1*. https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf
- [3] Rapps, S., & Weyuker, E. J. (1985). Selecting software test data using data flow information. *IEEE Transactions on Software Engineering*, SE-11(4), 367–375. <https://doi.org/10.1109/TSE.1985.232226>
- [4] Debian Security Team. (2008). *[DSA 1571-1] New openssl packages fix predictable random number generator*. CVE-2008-0166. <https://lists.debian.org/debian-security-announce/2008/msg00152.html>